ISE CS4422A
Week 7 Lecture 21 Writing Device Drivers in C

Mark Burkley

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Any questions from last lecture?

Agenda for this Session

- C structs
- Function pointers
- File operations
- Device nodes
- Memory
- Ioctls
- Practicals

Problem opening for append

- File mode r+ assumes file exists
- File mode w+ truncates a file
- File mode a+ always appends regardless of seek
  - https://stackoverflow.com/questions/10631862/fseek-does-not-work-when-file-is-opened-in-a-append-mode
- Soltuion?
  - Try r+ first and if that fails then try w+

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Kernel module C programming

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Declaring C structs

- A struct in C is a **collection** of variables
- In C++ and Java we also have classes which can contain **data** and **methods**
- A struct has a **definition** and a **declaration**
  - We must define the contents of the struct before we declare an instance of a struct
- We can **combine** the definition and the declaration
- In the declaration, we can include field **initialization**
- We can specify **which fields** we are initialising using a **dot notation**

```
struct student_str
{
        char name[10];
        int year;
};

struct student_str student=
{
        .name = "Mark",
        .year = 2025
};

struct teacher_str
{
        char name[10];
        char module[5];
}
teachers[10];
```

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Function pointers

- **Function pointers**, like variable pointers, are **addresses** in memory
  - Instead of pointing to data, they point to code
- They are declared by putting a * before the function name. We add brackets around the * and the name to void implying return type.
  - e.g. `int (*func)()`
- You can assign function names to function pointers.
  - e.g. `int (*printfunc)() = printf`
- Structs can contain function pointers and you can initialize the elements of the struct at runtime or statically
  - e.g. `struct ops_t ops = { .open = my_open };`
- This is a sort of OO approach used in a procedural language
  - In C++ (or Java) we would use **virtual methods** but we can't use C++ in kernel land

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

File operations

- When we **register** a device driver we need to tell the kernel what **operations** we will support
- We do this by **declaring** an instance of `struct file_operations` and **populating** the fields
- You can **initialize** the elements of the struct at runtime or statically
  - Static declaration is generally easier

```
#include <linux/fs.h>

ssize_t device_file_read (struct
file *, char *, size_t, loff_t
*);

static struct file_operations
simple_driver_fops =
{
    .owner = THIS_MODULE,
    .read = device_file_read
};
```

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Character or block device?

- A device driver can be a **character device** driver or a **block device** driver
- Block devices are for mass storage
- Character devices are for **byte streams**, so we will use this type
- Device drivers have a **major** number and a **minor** number.
  - The major number identifies the **module** that serves this particular device type
  - The minor number is the **instance** if there are multiple devices
- We can **register** a device by calling `register_chrdev()`

```
#define MAJOR 42

void my_reg()
{
        int register_chrdev(MAJOR,
                            "mydriver", fops);
}

void my_unreg()
{
        unregister_chrdev(MAJOR,
                "mydriver");
}
```

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Creating the device node

- The **interface** to a linux device driver from user space is through a **device node** file
- This is a file in the `/dev` directory that we create with the `mknod` command
- We must specify the file name, the type (c for character, b for block), the major and the minor number
- When we write user space apps we will interact with our driver by opening this file and performing operations such as `read()` and `write()`

```
# mknod /dev/mark c 42 1
```

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Init and exit

- A loadable module must have an **init** and an **exit** function
- For now, these functions can be empty
- Note the function prototypes must **exactly match** that defined in `<linux/init.h>` so `init()` returns an `int` and `exit()` returns `void`

```
#include <linux/init.h>
#include <linux/module.h>

static int mark_init(void)
{
    return  0;
}

static void mark_exit(void)
{
    return;
}

module_init(mark_init);
module_exit(mark_exit);
```

Building LKM

- To build projects composed of C (or other) modules we use the command line tool make
- The make tool looks for a file called `Makefile` to tell it what to do
- We can create this simple `Makefile` for our device driver
- We can replace driver.o with the name of our actual driver object file

```
KERNEL=/lib/modules/$(shell uname -r)/build
obj-m += driver.o
all:
        ${MAKE} -C ${KERNEL} M=$(PWD) modules
clear:
        ${MAKE} -C ${KERNEL} M=$(PWD) clean
```

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Adding functionality to your device driver

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Blocking in the kernel module

- We declare a `wait_queue` for any operations we want to block
- Any file operations that are called that may block and have the `O_NONBLOCK` flag set should not block
- Instead they should return – `EAGAIN` which means “please try again later”
- To wake a task that is blocking, we call `module_wake_up()`

```
#include <linux/fs.h>

static struct wait_queue *WaitQ = NULL;

static int module_open(struct inode *inode,
                       struct file *file)
{
   if (file->f_flags & O_NONBLOCK)
      return -EAGAIN;

   module_interruptible_sleep_on(&WaitQ);
}

void module_wake (void)
{
    module_wake_up(&WaitQ);
}
```

The proc file interface

- Proc files can be created in the `/proc` directory
- These files have content that is **created dynamically** whenever they are read
- `proc_create()` creates a proc file
- `remove_proc_entry()` removes the entry
- Any user space access to the proc file **invokes** the registered handler in the operations structure (next slide)
- Very useful method to find out what is happening **inside** a driver or **control** a driver's behaviour

```
static int my_proc_init(void)
{
    my_proc_a = proc_create(
        "tlc_proc_a", 0444, NULL,
        &tlc_proc_a_fops);
    if(tlc_proc_a==NULL) {
        printk(KERN_ALERT
               "Error: proc_create");
    }

    return 0;
}

static void my_proc_exit(void)
{
    remove_proc_entry("tlc_proc_a", NULL);
}
```

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Proc file operations

- We pass an **operations structure** to the call to `proc_create()`
- This structure contains **operations** to be performed on the proc file
  - Generally just **read** and/or **write** is sufficient
- **NOTE** the interface to create proc files changed in recent kernels. Most online examples are still showing the old method
  - The main difference is the use of the `struct proc_ops` instead of `struct file_operations`

```
#include <linux/fs.h>
#include <linux/proc_fs.h>

struct proc_dir_entry *my_proc;

static struct proc_ops my_proc_fops = {
        .proc_read=my_proc_read,
};

static ssize_t my_proc_read(struct file
*fp, char *buf, size_t len, loff_t * off)
{
        memcpy (buf, "hello", 5);
        return 5;
}
```

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

memory

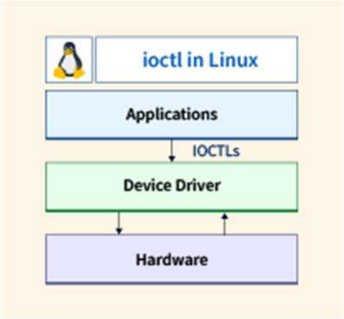
- Kernel memory is allocated and freed using `kmalloc()` and `kfree()`
- When allocating, the page type must be specified.
- `GFP_KERNEL` is the most common. It calls `get_free_pages()` and can block
- Exchanging memory with user space apps must be done with `copy_to_user()` and `copy_from_user()` since user space cannot access kernel memory

```
static char *my_get (char *user)
{
        int size = 10;
        char *ptr = kmalloc(10, GFP_KERNEL);
        copy_from_user(ptr, user, 10);
        return ptr;
}

static void my_put (char *user, char *ptr)
{
        copy_to_user(user, ptr, 10);
        kfree(ptr);
}
```

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

ioctl



- An **ioctl** is an escape or out-of-band method to communicate with a device driver
- Take an example where the character device node `/dev/serial` is the node for a **serial device**
- We can `read()` to fetch data from a remote sender or call `write()` to send data to a remote reader
- How do we change parameters such as baud rate?
- We can't use `write()` since anything we send to the driver will just get send over the serial link
- We need a way to "**escape**" our commands such as set baud rate so that they don't get sent over the serial link
- This is where we use the `ioctl()` function to provide commands
- We can define our own numeric codes for each driver that perform some system call that we define

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

ioctl kernel example

- An `ioctl()` call takes a memory pointer as a parameter
  - It can be a pointer to anything that the driver and app agree
  - It can be read (from kernel to user) or write (from user to kernel) or both or none
  - The macros `_IO`, `_IOR`, `_IOW` and `_IORW` define these
  - We also pass a “magic number” (which could be the MAJOR number) and a command number

```
#include <linux/ioctl.h>

#define WR_VALUE _IOW(42,1,int32_t*)
#define RD_VALUE _IOR(42,2,int32_t*)

static long etx_ioctl(struct file *file,
unsigned int cmd, unsigned long arg)
{
    switch(cmd) {
        case WR_VALUE:
                copy_from_user(&value
,(int32_t*) arg, sizeof(value))
                break;
        case RD_VALUE:
                copy_to_user((int32_t*)
arg, &value, sizeof(value))
                break;
        }
        return 0;
}
```

ioctl user app example

- The function `ioctl()` can be called from userspace with the same number as is defined for the kernel driver
- The parameter list is open, but the third parameter is typically a pointer to some data that driver and app have agreed to use

```
#include <sys/ioctl.h>

int main()
{
        int fd = open ("/dev/mark");
        ioctl (fd, WR_VALUE, "hello");
}
```

Questions and Feedback

- Comments or questions on this week's content?